

Tinza Marketplace — Documentation de l'Ingénierie et de l'Architecture

1. Concept de Base et Architecture du Système

Tinza Marketplace est un pipeline d'upload e-commerce multi-boutiques automatisé conçu pour orchestrer l'ingestion de lots de produits à haut volume. Le système traite automatiquement les dossiers Google Drive non structurés contenant des descriptions de produits (au format .docx ou Google Docs) et des ressources multimédias haute résolution (JPEG, PNG, HEIF, AVIF), exploite les grands modèles de langage (LLM) pour structurer le texte marketing et les options, et synchronise le tout vers des boutiques WooCommerce

1.1 Répartition des Composants

La plateforme repose sur une architecture de microservices modulaire et découplée, conçue pour une haute disponibilité, une isolation des pannes, une évolutivité horizontale et une faible charge opérationnelle :

- **Moteur de Workflow n8n:** Agit comme l'orchestrateur central du pipeline. Gère les webhooks Google Drive entrants, applique la mise en file d'attente basée sur la base de données, pilote la logique d'analyse LLM, coordonne la transformation des données à plusieurs étapes et garantit les mécanismes de nouvelle tentative du workflow. Son stockage et sa base de données d'état sont hébergés sur GCP (Cloud SQL).
- **API Proxy Flask (GCP):** Microservice Python autonome déployé sur GCP Cloud Run qui gère les tâches techniques lourdes et sécurisées pour n8n. Il s'occupe du chiffrement AES des clés WooCommerce, de la conversion et du redimensionnement des images en WebP, de l'extraction de texte à partir des documents Google Drive, et de la création accélérée des produits et de leurs déclinaisons sur WooCommerce via l'API REST.
- **Supabase (PostgreSQL & Stockage) :** Base de données PostgreSQL pour le routage multi-tenant, les webhooks, et la file d'attente. Stockage CDN pour les images WebP.
- **Google Cloud Platform (GCP):** Infrastructure hébergeant le microservice serverless, la base de données PostgreSQL gérée pour l'état du système n8n, Secret Manager pour la rotation des clés et les contrôles de sécurité IAM.

2. Architecture de la Base de Données

La couche de données isole les store owners et gère la file d'attente de manière transactionnelle. La logique métier réside sur Supabase (PostgreSQL), tandis que le stockage de l'état n8n est géré par Cloud SQL (GCP).

2.1 Schéma SQL (Supabase)

```
CREATE SCHEMA n8n_system;
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "pg_cron";

-- Stores Table: Holds multi-tenant vendor configurations & encrypted REST API keys
CREATE TABLE stores (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  store_name VARCHAR NOT NULL,
  wc_user TEXT NOT NULL,           -- AES-encrypted WooCommerce Consumer Key
  wc_pass TEXT NOT NULL,          -- AES-encrypted WooCommerce Consumer Secret
  drive_folder_id VARCHAR NOT NULL, -- Root Google Drive folder ID for vendor uploads
  default_status VARCHAR DEFAULT 'draft' -- Default WooCommerce status ('draft' | 'publish')
);

-- Webhook Channels Table: Tracks active Google Drive watch notifications
CREATE TABLE webhook_channels (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  store_id UUID REFERENCES stores(id) ON DELETE CASCADE,
  channel_id VARCHAR NOT NULL,
  resource_id VARCHAR NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  renewed_at TIMESTAMP,
  webhook_url VARCHAR
);

-- Processing Queue Table: Manages atomic task scheduling per product folder
CREATE TABLE processing_queue (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  store_id UUID NOT NULL,
  folder_id TEXT NOT NULL,         -- Product folder ID in Drive
  folder_name TEXT NOT NULL,       -- Product folder name
  drive_folder_id TEXT NOT NULL,
  default_status TEXT NOT NULL,
  status TEXT DEFAULT 'pending',   -- 'pending' | 'processing' | 'done' | 'exists' | 'failed'
  retry_count INT DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  CONSTRAINT processing_queue_store_folder_uniq UNIQUE (store_id, folder_id)
);

-- Indexes for performance & quick query routing
CREATE INDEX idx_stores_folder ON stores(drive_folder_id);
CREATE INDEX idx_channels_store ON webhook_channels(store_id);
CREATE INDEX idx_channels_channel ON webhook_channels(channel_id);
CREATE INDEX idx_processing_queue_status_created ON processing_queue(status, created_at ASC);

-- Concurrency Control Constraint: Enforces maximum ONE active worker per folder
CREATE UNIQUE INDEX idx_queue_active_folder
ON processing_queue (store_id, folder_id)
WHERE status IN ('pending', 'processing');
```

```
-- Automated Cleanup Job: Purges historical completed/failed queue records every 5
hours
SELECT cron.schedule(
  'purge-completed-queue-jobs',
  '0 */5 * * *',
  $$
    DELETE FROM public.processing_queue
    WHERE status IN ('done', 'failed', 'exists')
    AND updated_at < NOW() - INTERVAL '3 days';
  $$
);
```

2.2 Résumé du Modèle de Données

Nom de la Table	Objectif Principal
stores	Stocke les identifiants WooCommerce des store owners chiffrés.
webhook_channels	Enregistre les webhooks et indique s'ils sont toujours actifs ou pas par rapport aux notifications push de Google Drive.
processing_queue	Affiche la file d'attente ainsi que l'état d'exécution du produit.

3. Analyse du Backend Flask API

Le microservice Python Flask (déployé sur Cloud Run via Docker) agit comme un pont proxy. Il gère le traitement lourd des médias, les opérations cryptographiques, l'extraction de texte brut à partir de documents, et l'intégration de l'API REST WooCommerce.

3.1 Chiffrement et Cache

Le backend met en œuvre un chiffrement AES-256 CBC pour les identifiants vendeurs, une mise en cache en mémoire via LRU cache, et une connection pooling pour PostgreSQL et Supabase.

- **encrypt(text):** Chiffre les chaînes de caractères en utilisant AES-256.
- **decrypt_value(encrypted_text):** Déchiffre les clés API WooCommerce
- **@lru_cache get_store_credentials(store_id):** Met en cache en mémoire les identifiants d'API des stores pour réduire les requêtes DB.
- **@lru_cache get_wc_category():** Met en cache les recherches de catégories WooCommerce pour réduire les requêtes API.

3.2 Endpoint

A. Onboarding Endpoint (POST /onboard)

Reçoit les infos vendeurs depuis n8n, chiffre les clés et les enregistre dans la table 'stores', puis enregistre les webhooks dans la table 'webhook_channels'.

B. Text Extraction Endpoint (POST /extract-text)

Récupère les fichiers de documents directement depuis Google Drive à l'aide de l'API Google Drive v3. Fait automatiquement la différence entre les Google Docs natifs et les fichiers Microsoft Word (.docx). Renvoie la représentation en texte brut pour ingestion par le LLM.

C. Media Upload Endpoint (POST /upload-media)

Convertit (JPEG/HEIF/AVIF....) en WebP (qualité 80) et télécharge sur Supabase Storage.

D. Product Creation (POST /create-product)

- 1. Vérification Préalable du SKU:** Interroge l'API REST WooCommerce à partir du SKU du dossier. Si un produit existant est trouvé, il ignore la création et renvoie immédiatement le statut
- 2. Construction du Payload Unifié:** Assemble les données pour WooCommerce avec la description formatée en HTML, tags, la catégorie correspondante, les attributs et les URLs des images WebP.
- 3. Contournement de Conflit "Ghost SKU":** Crée d'abord le produit sans le SKU pour éviter les bugs de verrouillage de table. Il effectue ensuite une requête PUT secondaire pour attacher le SKU.

4. Workflows n8n

Workflow 1: Vendor Onboarding

Le formulaire n8n déclenche la création d'un dossier Google Drive, attribue les permissions au vendeur et met en place un webhook.

workflow 2: Product Listing

Partie 1: Webhook Listener

Un changement sur le Drive envoie un POST au webhook n8n. Le système filtre les dossiers (ignore _done) et ajoute les nouveaux dossiers dans la processing_queue. Il ignore aussi les dossiers qui y sont déjà insérés s'ils ont le même folder_id pour éviter les doublons.

Partie 2: Pipeline de Traitement

Un déclencheur planifié (Schedule Trigger) s'exécute toutes les 70 secondes pour traiter les tâches en attente de manière séquentielle :

- **1. Verrouillage et Réclamation de Ligne:** Exécute la commande SQL `SELECT FOR UPDATE SKIP LOCKED` pour réclamer exactement UNE tâche en attente dans `processing_queue` et verrouiller son statut sur `'processing'`.
- **2. Vérification du Contenu:** Inspecte les fichiers à l'intérieur du dossier pour s'assurer qu'au moins un fichier document (`.txt/.docx`) et un fichier média existent. Analyse le nom du dossier pour l'utiliser comme SKU du produit.
- **3. Extraction de Texte (Text Extraction) :** Appelle l'endpoint Flask `/extract-text` pour lire le texte brut depuis Google Drive.
- **4. Analyse Structurée par LLM (Structured LLM Parsing) :** Transmet le texte extrait à la chaîne LLM avec un prompt à schéma strict. Évalue le titre, les caractéristiques/spécifications en HTML, la catégorie unique correspondante, la tarification et la matrice de variantes.
- **5. JSON Sanitization:** Exécute un analyseur JS personnalisé avec des heuristiques d'auto-réparation JSON pour corriger les réponses LLM malformées.
- **6. Optimisation et Téléversement des Médias (Media Optimization & Upload) :** Télécharge chaque image depuis Google Drive, encode le contenu binaire en Base64, et envoie une requête POST à l'endpoint Flask `/upload-media` pour héberger les images optimisées sur le bucket Supabase et générer des liens WebP publics
- **7. WooCommerce Sync:** Assemble le payload JSON complet (métadonnées + images + variantes) et envoie une requête POST à l'endpoint Flask
- **8. Finalisation et Renommage (Finalization & Renaming) :** Met à jour le statut dans `processing_queue` vers `'done'` ou `'exists'`, puis appelle l'API Google Drive pour ajouter le suffixe `_done` au nom du dossier du produit.

workflow 4: Queue Timeout Recovery

Processus de récupération des délais de la file: S'exécute toutes les 5 minutes. Identifie les tâches bloquées au statut `'processing'` dont le champ `updated_at` date de plus de 3 minutes. Incrémente `retry_count` ; si le nombre de tentatives est inférieur à 9, réinitialise le statut à `'pending'`. Si le nombre de tentatives est supérieur ou égal à 9, passe le statut à `'failed'`.

workflow 5: Auto-Renewal

Surveillance du renouvellement automatique des webhooks: S'exécute toutes les 12 heures. Identifie les canaux de webhook Drive expirant dans les 24 heures. Réenregistre les canaux de suivi auprès de l'API Google Drive et met à jour les horodatages d'expiration dans Supabase.

5. Developer Onboarding, Maintainability & Scalability

5.1 Architectural Guarantees

- **Idempotency & Duplicate Prevention:** SKU existence pre-checks, active queue unique indexes, folder renaming with '_done' suffixes, and ON CONFLICT DO NOTHING clauses prevent duplicate WooCommerce listings under network retries or parallel triggers.
- **Scalability & Performance:** Decoupled architecture allows independent scaling. Cloud Run Flask containers scale horizontally; Supabase connection pooling isolates DB loads; n8n handles queue execution without blocking.
- **Security & Isolation:** All sensitive API keys are encrypted at rest with AES-CBC. Cloud Run environment secrets are managed via GCP Secret Manager.

5.2 Guide de démarrage rapide

Afin de maintenir ou faire évoluer le projet (pour plus d'informations techniques, veuillez consulter le fichier `README.md`):

1. Configuration de l'environnement local

```
# Local Environment Variables (.env)
AES_KEY=<64-char-hex-key>
DB_HOST=<supabase-db-host>
DB_PORT=5432
DB_NAME=postgres
DB_USER=postgres
DB_PASS=<db-password>
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_KEY=<service-role-key>
WC_BASE_URL=https://vendor-store.com
GOOGLE_APPLICATION_CREDENTIALS=service-account.json
```

2. Étapes de déploiement

```
# Deploy Flask API to Cloud Run
gcloud run deploy tinza-flask-api --source . --region us-central1 --port=8080
--env-vars-file .env --allow-unauthenticated

# Deploy n8n Container to Cloud Run
gcloud run deploy tinza-n8n --image=n8nio/n8n:2.6.4 --region us-central1 --
port=5678 --allow-unauthenticated
```